- Final Project
  - Final Project Proposal
    - 1. Abstract
    - 2. Motivation
    - 3. System Block Diagram
    - 4. Design Sketches
    - 5. Software Requirements Specification (SRS)
    - 6. Hardware Requirements Specification (HRS)
    - 7. Bill of Materials (BOM)
    - 8. Final Demo Goals
    - 9. Sprint Planning
  - Sprint Review #1
    - Last week's progress
    - Current state of project
    - Next week's plan
  - Sprint Review #2
    - Last week's progress
    - Current state of project
    - Next week's plan
  - MVP Demo
  - Final Report
    - 1. Video
    - 2. Images
    - 3. Results
      - 3.1 Software Requirements Specification (SRS) Results
      - 3.2 Hardware Requirements Specification (HRS) Results
    - 4. Conclusion
  - References

Review the assignment due date

Final Project

Team Number: 10

Team Name: Gimbalers

Team Member Name	Email Address
Cindy Liu	[cinl@seas.upenn.edu]
Mike Song	[chengyus@seas.upenn.edu]
Justin Yu	[justinyu@seas.upenn.edu]

GitHub Repository URL: https://github.com/upenn-embedded/final-project-s26-t10

GitHub Pages Website URL: [for final submission]*

Final Project Proposal

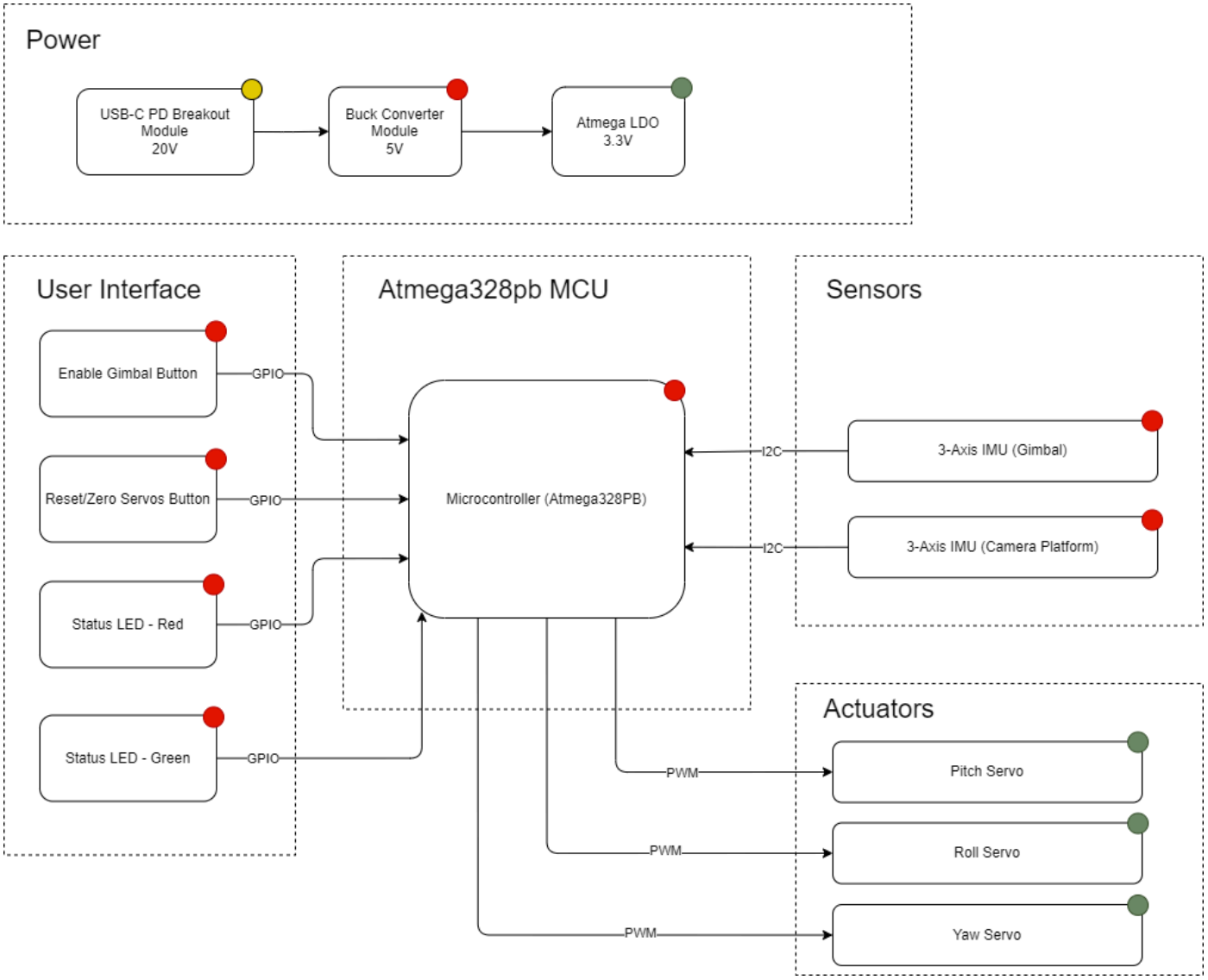
1. Abstract

This project is the design and implementation of a 3-axis camera gimbal system that uses servos to provide active stabalization for a camera. It uses an Inertial Measurement Unit (IMU) to monitor angular rotation as an input. Feedback is processed by an ATMega328PB. The microcontroller will output PWM signals, corrected by a PID control algorithm. Finally, the entire system will be powered by USB-C.

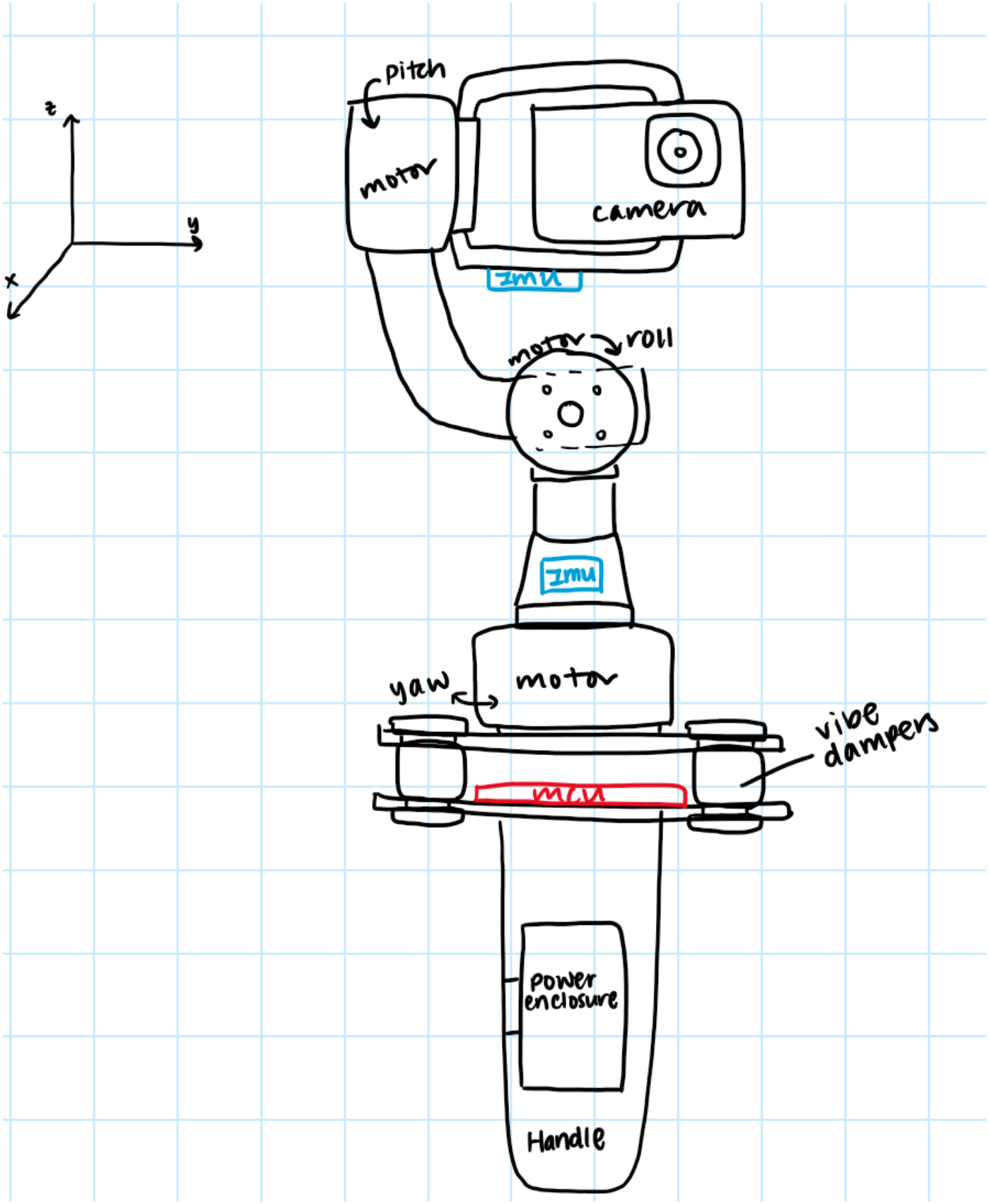
2. Motivation

Conventional camera gimbals are expensive, usually upwards of $200. Furthermore, they are usually designed specifically for industry grade cameras, leaving much to be desired for amateur videographers. This project gives an affordable option for those looking to try recording videos wth stabalization. By utilizing the widespread USB-C connector for power, this device can be used in many environments with a variety of power sources.

3. System Block Diagram



4. Design Sketches



5. Software Requirements Specification (SRS)

5.1 Definitions, Abbreviations

(IMU) - Inertial Measurement Device, gives linear acceleration and angular rate data
(Angular Rate) - degrees per second, as given by IMU (PID controller) Proportional-Integral-Derivative controller. Uses feedback to hit a target while reducing overshoot, in

this case to stabalize the axis after rotation. Uses integral and derivative of error to adjust output. (PWM) - pulse width modulation, produces a square wave output from the microcontroller to serve as an input to the servo.

5.2 Functionality

ID	Description
SRS-01	The IMU 3-axis angular rate will be measured with 16-bit depth every 100 milliseconds ±10 milliseconds.
SRS-02	The PID controller will recieve new inputs and produce PWM outputs for each servo every 100 milliseconds ±10 milliseconds, based on estimated error with an accumulator variable that sums IMU's angular rate every polling cycle.
SRS-03	The PWM Duty cycle that is output to the servos will be updated every 100 milliseconds ±10 milliseconds.
SRS-04	The gimbal will read the input of an "enable" button so it will stop stabalizing if the button is pressed.
SRS-05	The device can be reset to zeroed positions with the press of the "zero" button.
SRS-06	The device can be manually zeroed and the position can then be remembered with a long press of the "zero" button.
SRS-07	The user shall be able to determine the state of the device operation by viewing the status LED.

6. Hardware Requirements Specification (HRS)

6.1 Definitions, Abbreviations

MCU - Microcontroller Unit, compact integrated circuit that processes inputs and manages dedicated outputs and tasks. Camera Plate - Platform for GoPro/similar sized camera to rest on, controlled on 3 axes by the servos. Zeroing - Setting the default position for the camera plate, manually done by aligning the camera plate with the handle.

6.2 Functionality

ID	Description
HRS-01	USB-C PD module should deliver 20V output from an USB-C input.
HRS-02	Buck Converter Module should deliver stable 5V output.
HRS-03	ATmega328PB will function as the microcontroller for the device and also deliver 3.3V.
HRS-04	2 IMUs interfaced via I2C will deliver acceleration and position data to the MCU.
HRS-05	3 servos will control the pitch, roll, and yaw of the camera plate.
HRS-06	Two buttons that allows users to enable the gimbal function and zero the servos.
HRS-07	Servos combined with IMU inputs stabilize movement of camera plate in three degrees of freedom.
HRS-08	Servos combined with the IMU inputs can "lock" the camera onto a specific location/direction.

7. Bill of Materials (BOM)

https://docs.google.com/spreadsheets/d/1yj54xOVig_wChPn7wCOqRE96ZmyI3QeAy7MwsV0c4z8/edit?gid=2071228825#gid=2071228825

8. Final Demo Goals

Demonstrate a 3-axis (pitch and roll) active stabilization. Able to keep camera platform stable despite rapid movements. The system shall read angular velocity from an IMU, compute a stable angle estimate, and drive three servos. Independent PID controllers on each axis shall correct for angular error at 10Hz.

9. Sprint Planning

Milestone	Functionality Achieved	Distribution of Work
Sprint #1	Using an I2C Library, stabilize pitch, create handle and first servo mount	Mike will CAD the handle and servo mount prototype. Justin and Cindy will implement and tune PID for one axis.
Sprint #2	Move away from using I2C library, include second servo and begin tuning both axes in tandem, then moving on to include the third and last servo.	Mike will design and integrate the complete 3 servo mount and validate mechanical balance on all axes. Justin and Cindy will work on integration between the 3 axes and the PID loop.
MVP Demo	All axes stabilizing with servo actuation. Complementary filter producing stable angle estimates on all axes. PID visibly rejecting hand-induced disturbances. Mechanical assembly fully integrated.	Mike ensures full mechanical assembly is complete and camera platform is balanced. Cindy integrates both PID loops and validates stable closed-loop behavior on pitch and roll. Justin validates I2C driver and verifies clean IMU data.
Final Demo	Custom I2C driver, complementary filter, PID controllers, all written from scratch in register-level C. System visibly stabilizes camera footage under moderate hand disturbance at 10Hz.	Mike finalizes mechanical assembly, performs cable management, and captures demo footage. Justin and Cindy tunes PID gains on all axes, implements deadband, and optimizes control loop timing.

This is the end of the Project Proposal section. The remaining sections will be filled out based on the milestone schedule.

Sprint Review #1

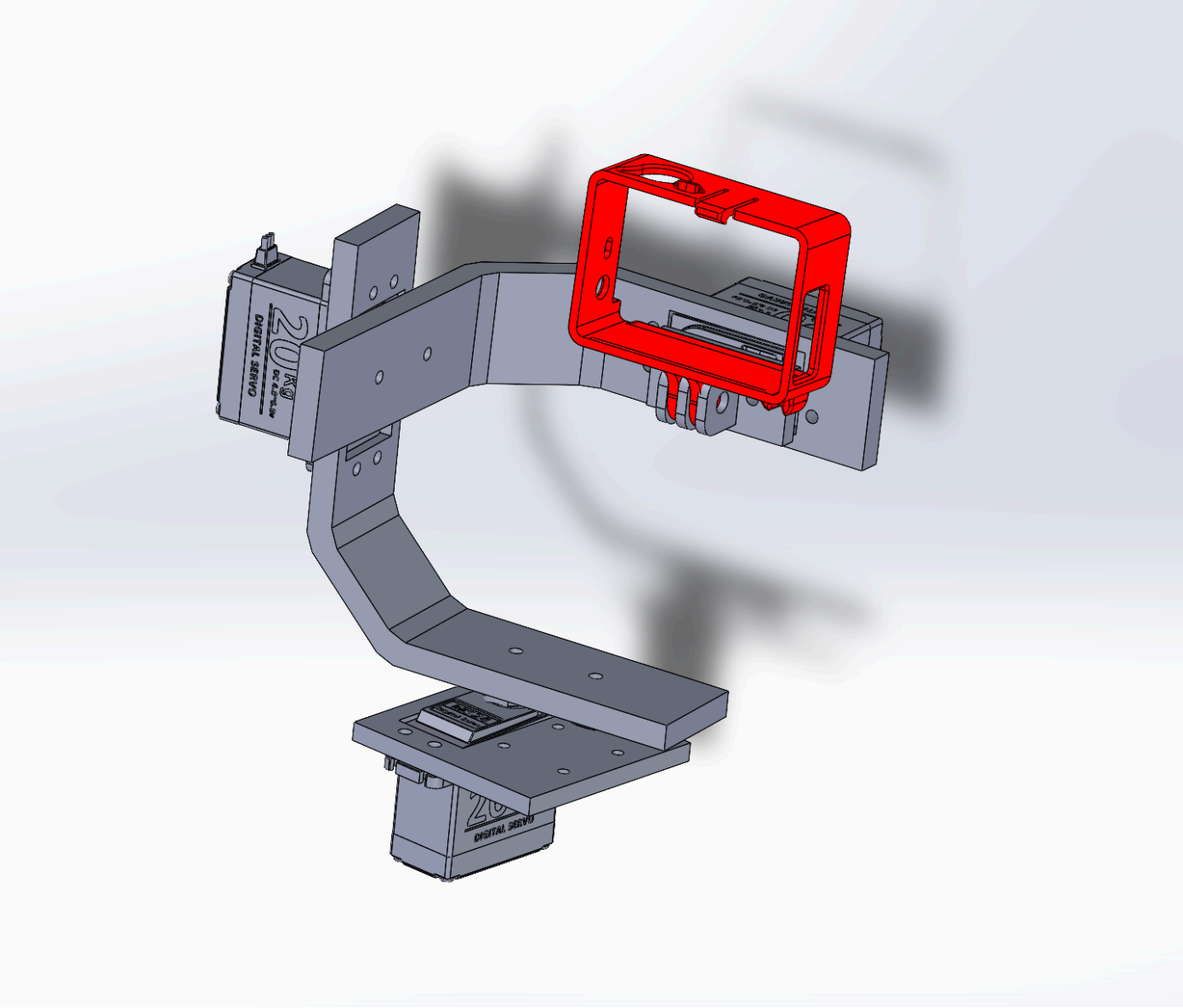
Last week's progress

In the past week, we started working on both the mechanical structure of the gimbal and the software for the IMU reading and servo control. We were able to get a preliminary assembly in CAD of the mates for each axis of rotation, as well as model the servo motors and camera frame. On the software side, we were able to use I2C to read the angle from the IMU, compute servo correction position, and generate an output PWM to stablize the system. Finally, we have also placed the order for our remaining parts.

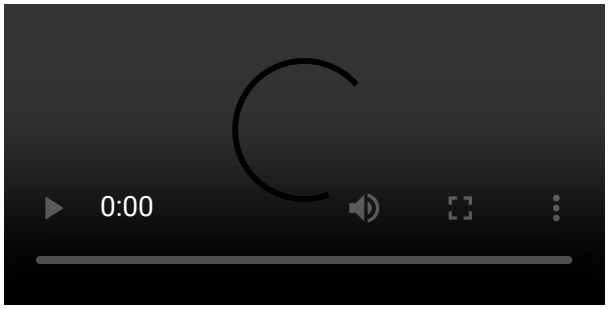
Current state of project

Code file: see main.C

CAD:



Video of motor stabalization in response to IMU:



Being able to get the motor to smoothly respond to rotation in one axis is a major step and provides a solid foundation for building on top of that. We foresee a lot more challenges in integrating PID into a multi-axis system since the axis and position will affect one another, but our current state is enough to ensure at least minimum functionality.

Next week's plan

Finish the CAD assembly and start 3D printing the parts at RPL. The CAD should take half a week and printing the parts may take longer, up to 1 week. (Mike)

Refine the algorithm for one-axis PID, and add in second IMU and servo. Expect to take up to 1.5 weeks. Start experimenting with power delivery with sample PD board from Detkin before actual power parts arrive, take around 1 week. (Justin and Cindy)

Sprint Review #2

Last week's progress

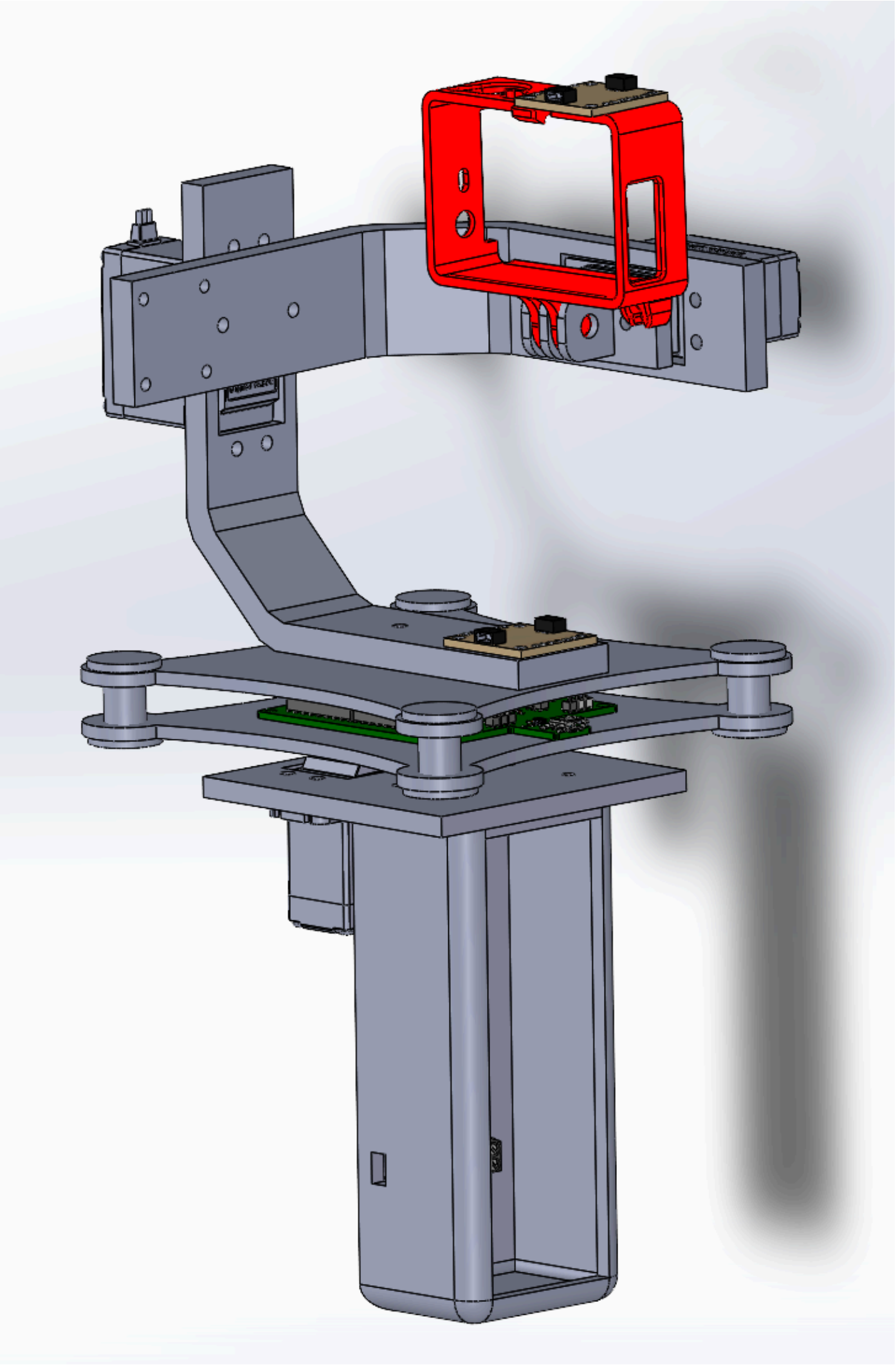
In the past week, we have made progress on both the mechanical assembly and the PID control. The parts of the assembly that were CADed last week have been 3d-printed such as the go-pro mount and its bracket, and we are testing to see if the parts fit. New parts were added to the CAD assembly, including the board mount and the handle. Some of our parts have also arrived, including a servo motor of our specified model that we used for testing the PID algorithm.

On the software side, we fine-tuned the kp parameter and dt of our PID control. Now the servo responds at a faster rate and the error between the IMU's rotation and the servo's rotation has been decreased. We then assembled the IMU, servo, and bracket together to create a 1-axis prototype.

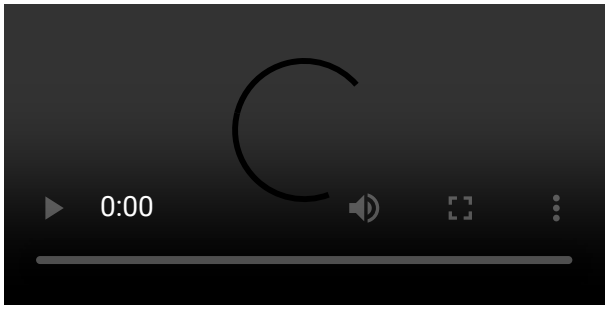
Current state of project

Code file: see main.C

CAD:



Video of 1-axis prototype (video might not load, please see on github online):



From the video, we see that the angle of the servo (where the go-pro camera mount will go) remains around the same angle relative to the ground.

Next week's plan

Finalize the CAD, print all the parts, and build a full assembly. This will allow us to know which parts need to be remeasured and fixed for the final print. (Assigned to Cindy and Mike, finish by MVP demo day). On the software side, the first goal is to integrate the other two servos so that we have a kp parameter tuned for three servos at once. (Assigned to Justin, finish by MVP demo day). If time permits, we can start adding I and D elements into the code to decrease error and oscillation.

MVP Demo

Final Report

Don't forget to make the GitHub pages public website! If you've never made a GitHub pages website before, you can follow this webpage (though, substitute your final project repository for the GitHub username one in the quickstart guide): https://docs.github.com/en/pages/quickstart

1. Video

2. Images

3. Results

3.1 Software Requirements Specification (SRS) Results

ID	Description	Validation Outcome
SRS-01	The IMU 3-axis acceleration will be measured with 16-bit depth every 100 milliseconds +/-10 milliseconds.	Confirmed, logged output from the MCU is saved to "validation" folder in GitHub repository.

3.2 Hardware Requirements Specification (HRS) Results

ID	Description	Validation Outcome
HRS-01	A distance sensor shall be used for obstacle detection. The sensor shall detect obstacles at a maximum distance of at least 10 cm.	Confirmed, sensed obstacles up to 15cm. Video in "validation" folder, shows tape measure and logged output to terminal.

4. Conclusion

References